4K Social Platform: Master Build Guide (Backend + App Integration)

> **Audience:** the senior coding agent working in this repo (Expo SDK 58 app + Express backend + Better Auth).
> **Mission:** replace every hard-coded social screen (home feed, "Who to Follow", profile, reflections/posts, tabs, notifications, streaks, ministries, highlights) with a **real, production-grade, fully functional** system: database, API, auth-aware authorization, media pipeline, notifications, search, moderation, seeding, tests, and the app-side data layer.
> **How to read this:** it is long on purpose. Sections 1 to 4 are rules and the plan. Sections 5 to 17 are the deep design. Section 18 onward is the delivery plan, testing, and the Definition of Done. **Follow the phases in order and respect the gates.**
> **Existing, working systems (auth, registration, etc.) must not break.** Build *on top* of them.

---

0. Operating rules for you (the agent)

1. **Audit before you build.** You have not seen my code in this prompt, and I haven't seen yours. Section 3 tells you what to inspect. Where this guide says "assumed", verify it and adapt. Where the repo already has a convention (folder layout, ORM, error format, lint rules), **follow the repo**, not this document, unless the repo's choice is actively harmful. In that case write an ADR (below) explaining the deviation.
2. **Keep a decision log.** Create `docs/adr/NNNN-title.md` for every non-trivial decision (context, options, decision, consequences). Keep each ADR short. This is how a human reviews your judgment.
3. **No hard-coded data in the app when you're done.** Every list, count, name, avatar, post, badge, suggestion, notification, streak, highlight and schedule on the screens described in §1 comes from the API. The only allowed static content: UI strings, icons, empty-state copy, and the design system.
4. **Never fake success.** No mutation in the UI is "done" until the server says so (optimistic UI is allowed, but it must roll back on failure).
5. **Security and privacy are features, not a phase.** Every endpoint is authenticated unless explicitly public, every resource access is authorized, every input validated, every list paginated.
6. **Small, reviewable, reversible changes.** One phase = a series of focused commits. Migrations are forward-only and tested. Feature-flag anything risky.
7. **Do not touch Better Auth's core tables or flows** (`user`, `session`, `account`, `verification`, and their columns). Extend via your own tables (1:1 `profiles`) and Better Auth's documented

extension points (`additionalFields`, plugins, hooks). Don't write raw inserts into Better Auth tables, not even in seeds (§15).

8. **Verify, don't assume.** Run the typecheck, linter, tests, migrations and the app. **Do not claim a behavior you didn't run.** In your final report, separate "verified by running" from "reasoned but not run".
9. **Performance is part of correctness.** Every list endpoint must have an index-backed query plan (`EXPLAIN` it), and no N+1 queries.
10. **Consistency with the earlier guides.** The app already has (or will have) the profile zoom transition, the long-press avatar preview, swipeable top tabs and local reminders. The API you build must supply what those need: avatar **thumbnails + a ≥1000 px variant + blurhash/thumbhash**, **author summaries embedded in every list item** (so the profile can open instantly from a seed), stable IDs, and tiny route params.

---

1. What the screens show → what must be real

I analyzed the four screenshots of the app ("4K", a university fellowship community: "4 Kilo Fellowship", campus ministry life, Addis Ababa). Treat the list below as the **functional spec**. Anything not listed here but obviously implied (empty/error states, pagination, offline) is also required.

1.1 Home

- **Top bar:** the user's own avatar (opens their profile/menu), app logo, a **streak chip** (flame + current streak, e.g. "1"), a **notification bell with an unread badge** (e.g. "2").
- **Content list ("videos"):** cards with thumbnail + play icon, title, short description (truncated), duration. These are curated media items (sermons, worship nights, orientation videos). A `content_items` module, not posts.
- **"Who to Follow" carousel** with a **View all** button. Each card: avatar with an **online dot**, display name, fellowship role ("Worship Leader", "Prayer Coordinator" in the accent color), department + year ("Software Eng. • 4th Yr"), a **"N mutual peers" chip**, a **Follow** button, and a **dismiss** ✕.
- Bottom tabs (Home, Library/Bible, Community, Gifts/Giving, Profile): out of scope except their data needs.

1.2 Profile

- **Cover image**, **avatar with a colored ring**, online dot, **verified badge** next to the name, `@username`.
- **Action buttons:** share/send, edit/compose, **Follow** (own profile shows "Edit profile" instead).
- **Badges row:** ministry ("Worship Team"), academic ("Software Engineering · 4th Year"), **streak ("32d Streak")**.
- **Bio**, **location** ("4 Kilo Fellowship Hall"), **joined date** ("Joined September 2022"), **external link** ("@abenezer_worship", with a messaging/social glyph).

- **Counts:** `Following`, `Connects`, `Reflections` (see §1.5 for the terminology decision).
- **"Followed by X, Y and N others"** with 3 small avatars: the *mutual-follow social proof* for the viewer.
- **Highlights row** (circle covers with labels: Sunday, Worship, Camp '25, Outreach, Testimonies): curated collections, like Instagram highlights.
- **Tabs:** *Reflections* (posts), *Media*, *Ministry*, *Likes*.
  - Reflections: Twitter-style post cards: author, verified badge, relative time, text, optional image, and counters for **reply · repost · like · bookmark**.
  - Ministry tab: ministries the person belongs to (e.g. "Worship Team: Active Fellowship Ministry"), and fellowship events/schedule ("Weekly Fellowship Schedule: Wednesday Midweek Prayer 5:30 PM, Sidist Kilo").
- **Collapsing header** with a back arrow, name + ministry subtitle, and share.
- **Pull-to-refresh** (a "Refreshing…" banner is visible).

1.3 Required "real" capabilities that follow from the screens

Profiles (read/edit), follow graph (follow/unfollow, suggestions, mutuals, dismiss), posts (create/edit/delete, media), replies/threads, reposts/quotes, likes, bookmarks, feeds (Following + For You), notifications (in-app + push, unread count), media upload pipeline, search (people, posts, hashtags), streaks (daily check-in), ministries + events, highlights, content library (videos), presence ("online dot"), moderation (report/block/mute), account lifecycle (incl. **account deletion**), admin tooling, analytics/observability.

1.4 Hard-coded things to hunt down

Search the app for literals like `Abenezer`, `Hiwot`, `28 mutual`, `32d`, `Worship Team`, `210`, `480`, mocks in `data/`, `mock`, `fixtures`, `constants`, inline arrays inside components, and `useState([...])` initial lists. List every one in your audit report (§3) and the endpoint that replaces it.

1.5 Terminology and assumptions (verify, then record in an ADR)

UI term	Domain model (recommended)	Note
Reflections	`posts` where `kind in ('post','quote')` authored by the user (replies live in a separate tab/section if desired)	UI copy can say "Reflections"; DB and API stay generic (`post`).
Connects	**Followers count** (people who follow this user)	The screenshot shows "Following / Connects / Reflections". "Connects" is most plausibly the followers label. **If the existing code or product defines it**

UI term	Domain model (recommended)	Note
		as *mutual* connections instead, follow that and adjust the counter logic. Record the decision.
Mutual peers (suggestion card)	Number of people the *viewer follows* who also follow the candidate (and optionally shared ministry/department)	Define precisely in §9.
Online dot	`profiles.last_seen_at` within the last ~2 min **and** the user allows showing presence	Privacy toggle required.
Ring on avatar	A boolean/enum derived from state (e.g. has active highlight, ministry leader, or "live")	Ask: what does the existing UI code use it for? Follow it.
Streak	Count of consecutive **local calendar days** with a qualifying action (check-in/app open/devotional read), computed in the user's timezone (default `Africa/Addis_Ababa`)	Server-authoritative.

---

2. Principles of a production social backend (read this; it drives every design below)

Real systems (X/Twitter, Instagram, Threads, Discord, Reddit) look different at scale, but all share these ideas. Your implementation must reflect them **at the right scale for this app** (thousands to low hundreds of thousands of users, one region, a small team): a **modular monolith**, not microservices.

1. **The social graph is the core dataset.** Everything personal (feed, suggestions, notifications, privacy) is a function of *who follows/blocks/mutes whom*. Model it as plain, indexed edge tables (`follows`, `blocks`, `mutes`) with composite primary keys, and keep counts denormalized.
2. **Read-heavy, write-light.** Feeds and profiles are read 100 to 1000× more than written. So: denormalize counters, index for the read path, cache hot reads, paginate everything, and keep writes small, transactional and idempotent.
3. **Cursor pagination, never offset.** Offsets drift when new items arrive and get slower as the page number grows. Use keyset (cursor) pagination on `(created_at, id)` or a time-ordered ID.
4. **Idempotency everywhere.** Mobile networks retry. `PUT /follow` twice = one follow. `POST /posts` with the same `Idempotency-Key` = one post. Likes/follows/bookmarks are *state* (PUT/DELETE), not *events* (POST).

5. **Counters are denormalized and eventually reconciled.** Updating `like_count` in the same transaction as the `likes` insert is correct and fast. A nightly job recomputes from source-of-truth to repair drift.
6. **The outbox pattern for side effects.** When a transaction commits a like/follow/reply, it also writes an `outbox_events` row *in the same transaction*. A worker later turns events into notifications, push messages, feed fan-out, search indexing and analytics. That way a crash can never create a like without its notification (or vice versa) and the HTTP request stays fast.
7. **Feeds are a pipeline:** *candidate generation → filtering (blocks/mutes/deleted/visibility) → ranking → mixing/dedupe → hydration (authors, media, viewer state)*. Start with a simple, explainable pipeline and keep the stages separate so each can be improved independently.
8. **Hydration is batched.** Return list items with embedded author summaries, media, counts, and **viewer state** (`liked`, `reposted`, `bookmarked`, `following`) computed with set-based queries (`WHERE post_id = ANY($ids)`), never per-row queries.
9. **Authorization is data-aware.** "Can viewer V see post P?" depends on author privacy, blocks, deletion, and moderation state. Centralize it in one policy layer used by every endpoint (and every list query via SQL filters).
10. **Media never flows through your API server.** Clients upload straight to object storage via presigned URLs. Your server validates, processes (resize, strip EXIF, blurhash) in a worker, and serves via CDN.
11. **Trust & Safety is built in:** report, block, mute, rate-limit, spam heuristics, moderation queue, audit log, content policy, **account deletion**. App stores require UGC apps to provide reporting, blocking and filtering, and a way to delete the account. (Check Apple App Store Review Guidelines §1.2 and §5.1.1(v) and Google Play's UGC policy.)
12. **Observability from day one:** request IDs, structured logs, metrics, error tracking, slow-query logging. You can't fix what you can't see.
13. **Boring technology wins.** Postgres for relational truth, Redis for queues/rate-limits/cache, S3-compatible storage for media. Add search engines, graph DBs, Kafka *only* when measurements demand it.

---

3. Phase 0: audit and report (do this first; produce a written report)

Inspect and report on **all** of the following. Put the report in `docs/audit.md` and summarize in your reply.

Backend

1. Runtime/framework versions: Node version, **Express 4 or 5** (this changes the Better Auth catch-all route pattern, see §5.2), ESM vs CJS (**Better Auth requires ESM**), TypeScript setup.
2. How Better Auth is configured: adapter/database, plugins enabled (e.g. Expo plugin, email/password, OAuth, `admin`, `username`, etc.), `additionalFields`, `trustedOrigins`, session

settings, email verification, rate limiting.

3. The **database engine and ORM/query layer** (Postgres + Drizzle/Prisma/Kysely? SQLite? MongoDB?). Migration tooling and how migrations are run.
4. Existing route structure, middleware order (**is `express.json()` mounted before or after the Better Auth handler?** it must be after), error handling, validation approach, logging.
5. Env/config management, secrets, CORS config, deployment target (Render/Railway/Fly/VPS/other), CI.
6. Existing tests and how they run.
7. Any social tables or endpoints that already exist (reuse, don't duplicate).

App (Expo)
8. Data fetching approach (TanStack Query? SWR? fetch + `useEffect`? Redux?) and the HTTP client. How session cookies/tokens are attached to requests.
9. Navigation structure and the screens in §1. For each, which components render hard-coded data.
10. Image component, list component (FlatList vs FlashList), state management, storage layer (MMKV/AsyncStorage/SecureStore), i18n, theme tokens.
11. Existing hooks/utilities you'll reuse (e.g. `useProfile`, `ProfileLink`, `LongPressAvatar`, reminders).

Output of the audit

- A table: *Screen → hard-coded sources → target endpoint(s) → phase*.
- A list of risks (e.g. SQLite in production, no migrations, `express.json()` before auth, no input validation).
- The **stack decisions** (§4) you're adopting, and each deviation as an ADR.
- **A written plan with phase gates.** Don't start Phase 1 until the audit is written.

> **STOP CONDITION:** if the existing database is not Postgres (e.g. SQLite/MongoDB) and holds production data, do **not** migrate silently. Write an ADR with options (keep it vs add Postgres for the social domain vs migrate with a tested plan) and **default to Postgres for the social domain**, because the social graph, feed queries, counters and full-text search are relational. Keep Better Auth's existing storage unless there's a compelling reason; auth users are referenced by id.

---

4. Target stack (defaults; follow the repo where it already chose)

Concern	Default	Notes
Language	**TypeScript** (strict), **ESM**	Better Auth does not support CommonJS.

Concern	Default	Notes
API	**Express** (keep)	Express 5 if already on it. Keep module boundaries so the framework is swappable.
DB	**PostgreSQL** (≥ 15; 16+ preferred)	Extensions: `citext`, `pg_trgm`, `unaccent` (search).
ORM/queries	**Drizzle ORM** + SQL where needed (or the repo's existing ORM)	Typed schema, SQL migrations checked into git. Raw SQL for the feed/suggestion queries is fine.
Validation	**Zod** at every boundary (HTTP in/out, env, job payloads)	Derive TS types from schemas.
Contracts	**Shared package** (`packages/contracts` or a `shared/` folder) exporting Zod schemas + types used by API and app; optionally generate **OpenAPI** from them	Single source of truth, no drift.
Queue/jobs	**Redis + BullMQ** (workers in the same repo, separate process)	Outbox relay, notifications, media processing, reconcile jobs, streak rollups.
Cache / rate limit	**Redis**	`rate-limiter-flexible` or `express-rate-limit` with Redis store.
Object storage	**S3-compatible** (Cloudflare R2 / AWS S3; **MinIO** in dev)	Presigned PUT/POST uploads, CDN in front.
Image processing	**sharp** in a worker	Resize variants, strip EXIF, compute blurhash/thumbhash.
Video	**Managed** (Mux / Cloudflare Stream) *or* keep external URLs for the curated library	Don't transcode video yourself.
Push	**Expo Push Service** via `expo-server-sdk`	Handle tickets and receipts, prune dead tokens.
Auth	**Better Auth** (existing)	Session via cookie, mobile via the Expo plugin and `authClient.getCookie()`.
Logging	**pino** + request IDs; **Sentry** for errors (API + app)	Redact PII/secrets.

Concern	Default	Notes
Testing	**Vitest** + **Supertest** + real Postgres (Docker/Testcontainers); **Maestro** for mobile E2E	No mocking the database in integration tests.
App data layer	**TanStack Query v5**	Infinite queries, optimistic mutations, persisted cache.
App lists	**FlashList** for feeds, **expo-image** for images	Consistent with earlier guides.
Local dev	**docker-compose**: postgres, redis, minio	One command to start.

Docs to read before you code (the docs win over this guide):

Topic	Link
Better Auth: Express integration	https://better-auth.com/docs/integrations/express
Better Auth: Expo integration	https://better-auth.com/docs/integrations/expo
Better Auth: docs index (hooks, `additionalFields`, plugins, rate limit, delete user)	https://better-auth.com/docs
Expo push: sending notifications (SDK 58)	https://docs.expo.dev/push-notifications/sending-notifications.md
Expo push: setup & credentials	https://docs.expo.dev/push-notifications/push-notifications-setup.md
Expo Router API routes (for reference)	https://docs.expo.dev/router/reference/api-routes
TanStack Query: infinite queries	https://tanstack.com/query/latest/docs/framework/react/guides/infinite-queries
TanStack Query: optimistic updates	https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates
TanStack Query: persistence	https://tanstack.com/query/latest/docs/framework/react/plugins/persistQueryClient

Topic	Link
Drizzle ORM	https://orm.drizzle.team/docs/overview
Zod	https://zod.dev
BullMQ	https://docs.bullmq.io
`expo-server-sdk-node`	https://github.com/expo/expo-server-sdk-node
sharp	https://sharp.pixelplumbing.com
ThumbHash / BlurHash	https://github.com/evanw/thumbhash , https://blurha.sh
PostgreSQL: full-text search, `pg_trgm`, indexes	https://www.postgresql.org/docs/current/
RFC 9457 (Problem Details for HTTP APIs)	https://www.rfc-editor.org/rfc/rfc9457
OWASP API Security Top 10 (2023)	https://owasp.org/API-Security/
Apple App Store Review Guidelines (UGC §1.2, account deletion §5.1.1(v))	https://developer.apple.com/app-store/review/guidelines/
Faker (seeded data)	https://fakerjs.dev
Maestro (mobile E2E)	https://maestro.mobile.dev

---

5. Backend architecture

5.1 Modular monolith layout (adapt names to the repo)

```
apps/api/src/
  app.ts                  // builds the Express app (no listen); used by tests
  server.ts               // listen, graceful shutdown
  config/                 // env.ts (Zod-validated), constants
  platform/               // cross-cutting: logger, db, redis, queue, storage,
```

```
errors, http helpers
  auth/                   // better-auth instance, requireAuth middleware, viewer
context
  modules/
    profiles/             // profile read/update, username rules, links, presence
    follows/              // follow/unfollow, followers, following, mutuals,
suggestions, dismissals
    posts/                // CRUD, replies, quotes, reposts, edit window, soft
delete
    interactions/         // likes, bookmarks
    feed/                 // following feed, for-you feed, ranking, hydration
    media/                // upload intents, assets, processing callbacks
    notifications/        // in-app, aggregation, preferences, push devices
    search/               // people, posts, hashtags
    streaks/              // check-in, rollups
    ministries/           // ministries, members, events/schedule
    highlights/           // highlight collections
    content/              // curated video/library items for the home list
    moderation/           // reports, blocks, mutes, admin queue, audit log
    account/              // deletion, export, preferences
  workers/                // BullMQ processors (outbox relay, push, media,
reconcile)
  db/
    schema/               // tables
    migrations/
    seed/                 // see §15
  tests/
packages/contracts/       // Zod schemas + TS types shared with the Expo app
```

Each module exposes: `routes.ts` (HTTP + auth + validation), `service.ts` (business rules, transactions), `repo.ts` (SQL/ORM only), `policy.ts` (authorization), `dto.ts` (serializers). **Rules:** routes never touch the DB, repos never contain business rules, services never read `req` / `res`, and modules talk to each other through services (never other modules' tables).

5.2 Better Auth integration points (verified against the docs; re-verify versions)

- **Handler mount order matters.** Mount the Better Auth handler **before** body parsers. `express.json()` before it makes auth requests hang. Express 4 pattern: `app.all("/api/auth/*", toNodeHandler(auth))`. **Express 5:** `app.all("/api/auth/*splat", toNodeHandler(auth))`. Better Auth needs **ESM**.
- **Getting the viewer in your own routes:**

```
// auth/requireAuth.ts
import { fromNodeHeaders } from "better-auth/node";
import { auth } from "./auth.js";
```

```
export async function loadViewer(req, _res, next) {
  const session = await auth.api.getSession({ headers: fromNodeHeaders(req.headers)
});
  req.viewer = session ? { id: session.user.id, session: session.session, user:
session.user } : null;
  next();
}
export function requireAuth(req, res, next) {
  if (!req.viewer) return res.status(401).type("application/problem+json").json({
type: "about:blank", title: "Unauthorized", status: 401 });
  next();
}
```

- **Expo client:** use `@better-auth/expo` (`expoClient({ scheme, storagePrefix, storage: SecureStore })`). For your own API calls, attach the session cookie from `authClient.getCookie()` as the `Cookie` header (see the Expo integration doc for the exact recommended fetch options such as `credentials`). Put this in **one** `apiFetch` wrapper, never scattered.
- `trustedOrigins` must include the app scheme (`myapp://`) and, in development only, the `exp://` patterns.
- **CORS:** the mobile app doesn't need CORS, but any web/admin client does. Use an explicit allow-list with `credentials: true`.
- **Extending users:** do **not** add social columns to Better Auth's `user` table. Create `profiles` (1:1, `user_id` PK/FK, `ON DELETE CASCADE`). Create the profile **in a Better Auth database hook / after-signup step** (check the docs' "hooks"/"database hooks") so it exists for every account, including OAuth sign-ups. Make creation idempotent (`INSERT … ON CONFLICT DO NOTHING`) and backfill existing users in a migration.
- **Usernames:** either Better Auth's `username` plugin (if enabled) or your own `profiles.username`. **One source of truth.** Check what exists in the audit.
- **Roles:** for admin/moderator use Better Auth's `admin` plugin if already enabled, or a `roles` column on `profiles` checked server-side. Never trust a role sent by the client.
- **Account deletion:** wire the app's "Delete account" to Better Auth's delete-user flow *and* a cleanup job for your domain data (§13).

5.3 Cross-cutting platform pieces (build these in Phase 1)

1. **Config:** `env.ts` parses `process.env` with Zod and **fails fast** on boot. No `process.env` reads elsewhere.
2. **Errors:** a typed `AppError(code, status, detail)` hierarchy, and **one** error middleware producing **RFC 9457** `application/problem+json`: `{ type, title, status, detail, code, requestId, errors? }`. Never leak stack traces or SQL.

3. **Validation middleware:** `validate({ body, query, params })` with Zod. Reject unknown keys (`.strict()`) to prevent mass assignment.
4. **Request context:** `requestId` (accept `X-Request-Id` or generate), a `pino` child logger on `req.log`, timing logs, and viewer id in logs. Redact `authorization`, `cookie`, tokens, emails.
5. **Pagination helpers:** opaque cursor encode/decode (base64url JSON `{ t, id }`), `limit` default 20, **max 50**, and a response envelope `{ items, nextCursor, hasMore }`.
6. **Idempotency middleware** for non-idempotent `POST`s (create post, upload intent, report): header `Idempotency-Key`, stored in `idempotency_keys(user_id, key, request_hash, response_status, response_body, created_at)` with a unique `(user_id, key)` and a TTL (24 h). Replays return the original response; a key reused with a different payload returns `422`.
7. **Rate limiting:** global (per IP) + per-viewer + per-route budgets (see §12). Return `429` with `Retry-After`. Better Auth has its own limiter for auth routes; don't duplicate it.
8. **Transactions:** a `withTx(fn)` helper; services decide transaction boundaries.
9. **Outbox:** `outbox_events(id, type, aggregate_id, payload jsonb, created_at, processed_at, attempts, last_error)` written in the same tx as the domain change; an `outbox-relay` worker polls (`FOR UPDATE SKIP LOCKED`), enqueues BullMQ jobs and marks processed. Handlers must be **idempotent** (dedupe by event id).
10. **Health & lifecycle:** `/healthz` (process up), `/readyz` (DB + Redis reachable), graceful shutdown on `SIGTERM` (stop accepting, drain, close pools).
11. **Versioning:** all routes under `/api/v1`. Add `X-Min-App-Version` / a `GET /api/v1/meta` returning `{ minAppVersion, serverTime, features }` so the app can force-update and sync clocks.

---

6. Data model (PostgreSQL)

> Reference DDL. Translate into the repo's ORM/migrations. Use **time-ordered IDs (UUIDv7 or ULID)** generated in app code for all domain tables so the primary key itself sorts by creation time (great for cursors and index locality). Match the type of `user.id` used by Better Auth (commonly `text`) for user foreign keys. All timestamps are `timestamptz` in UTC.

```
create extension if not exists citext;
create extension if not exists pg_trgm;
create extension if not exists unaccent;

-- 1:1 with Better Auth "user". Never alter the auth tables themselves.
create table profiles (
  user_id        text primary key references "user"(id) on delete cascade,
  username       citext not null unique check (username ~ '^[a-z0-9_]{3,20}$'),
  display_name   text not null check (char_length(display_name) between 1 and 60),
  bio            text check (char_length(bio) <= 300),
  location       text check (char_length(location) <= 80),
  department     text,                -- "Software Engineering"
```

```
  study_year     smallint check (study_year between 1 and 8),
  fellowship_role text,                -- "Worship Leader" (shown on suggestion
cards)
  avatar_asset_id uuid,                -- -> media_assets
  cover_asset_id  uuid,
  is_verified    boolean not null default false,   -- admin-set only
  visibility     text not null default 'public' check (visibility in
('public','followers')),
  show_presence  boolean not null default true,
  timezone       text not null default 'Africa/Addis_Ababa',
  last_seen_at   timestamptz,
  followers_count int not null default 0 check (followers_count >= 0),  -- UI label
"Connects"
  following_count int not null default 0 check (following_count >= 0),
  posts_count     int not null default 0 check (posts_count >= 0),      -- UI label
"Reflections"
  username_changed_at timestamptz,
  created_at     timestamptz not null default now(),
  updated_at     timestamptz not null default now()
);
create index profiles_trgm_username on profiles using gin (username gin_trgm_ops);
create index profiles_trgm_name on profiles using gin (lower(unaccent(display_name))
gin_trgm_ops);

create table profile_links (               -- "@abenezer_worship" with a platform
glyph
  id uuid primary key, user_id text not null references profiles(user_id) on delete
cascade,
  kind text not null check (kind in
('telegram','instagram','youtube','tiktok','website')),
  handle text, url text not null, position smallint not null default 0
);

-- Social graph
create table follows (
  follower_id text not null references profiles(user_id) on delete cascade,
  followee_id text not null references profiles(user_id) on delete cascade,
  status text not null default 'accepted' check (status in ('accepted','pending')),
  created_at timestamptz not null default now(),
  primary key (follower_id, followee_id),
  check (follower_id <> followee_id)
);
create index follows_followee_idx on follows (followee_id, created_at desc,
follower_id);
create index follows_follower_idx on follows (follower_id, created_at desc,
followee_id);

create table blocks (blocker_id text, blocked_id text, created_at timestamptz
```

```
default now(),
  primary key (blocker_id, blocked_id), check (blocker_id <> blocked_id));
create index blocks_blocked_idx on blocks (blocked_id, blocker_id);
create table mutes  (muter_id text, muted_id text, created_at timestamptz default
now(),
  primary key (muter_id, muted_id), check (muter_id <> muted_id));
create table follow_dismissals (user_id text, dismissed_id text, created_at
timestamptz default now(),
  primary key (user_id, dismissed_id));      -- the ✕ on "Who to Follow"

-- Posts ("Reflections"), replies, reposts and quotes share one table
create table posts (
  id uuid primary key,                       -- UUIDv7/ULID
  author_id text not null references profiles(user_id) on delete cascade,
  kind text not null check (kind in ('post','reply','repost','quote')),
  body text check (char_length(body) <= 500),
  parent_id uuid references posts(id),       -- direct parent for replies
  root_id   uuid references posts(id),       -- thread root for replies
  repost_of_id uuid references posts(id),    -- kind='repost'
  quoted_id    uuid references posts(id),    -- kind='quote'
  lang text,
  visibility text not null default 'public' check (visibility in
('public','followers')),
  like_count int not null default 0 check (like_count >= 0),
  reply_count int not null default 0 check (reply_count >= 0),
  repost_count int not null default 0 check (repost_count >= 0),
  quote_count int not null default 0 check (quote_count >= 0),
  bookmark_count int not null default 0 check (bookmark_count >= 0),  -- private,
not exposed
  created_at timestamptz not null default now(),
  edited_at timestamptz,
  deleted_at timestamptz,
  check ((kind='repost') = (repost_of_id is not null)),
  check ((kind='quote')  = (quoted_id    is not null)),
  check ((kind='reply')  = (parent_id    is not null)),
  -- "body or media required" (except reposts) is enforced in the service layer
inside the create transaction
  check (kind = 'repost' or body is null or char_length(btrim(body)) > 0)
);
create index posts_author_time on posts (author_id, created_at desc, id desc) where
deleted_at is null;
create index posts_parent_time on posts (parent_id, created_at, id) where deleted_at
is null;
create index posts_root_time   on posts (root_id,   created_at, id) where deleted_at
is null;
create unique index posts_one_repost_per_user on posts (author_id, repost_of_id)
where kind='repost' and deleted_at is null;
create index posts_body_fts on posts using gin (to_tsvector('simple',
```

```
coalesce(body,'')));

create table post_media (post_id uuid references posts(id) on delete cascade,
  asset_id uuid not null, position smallint not null, alt_text text check
(char_length(alt_text) <= 420),
  primary key (post_id, position));

create table likes (user_id text, post_id uuid references posts(id) on delete
cascade,
  created_at timestamptz default now(), primary key (user_id, post_id));
create index likes_post_idx on likes (post_id, created_at desc);
create index likes_user_idx on likes (user_id, created_at desc, post_id);     --
"Likes" tab
create table bookmarks (user_id text, post_id uuid references posts(id) on delete
cascade,
  created_at timestamptz default now(), primary key (user_id, post_id));
create index bookmarks_user_idx on bookmarks (user_id, created_at desc, post_id);

create table hashtags (id bigserial primary key, tag citext unique not null);
create table post_hashtags (post_id uuid references posts(id) on delete cascade,
hashtag_id bigint references hashtags(id), primary key (post_id, hashtag_id));
create table post_mentions (post_id uuid references posts(id) on delete cascade,
user_id text references profiles(user_id) on delete cascade, primary key (post_id,
user_id));

-- Media
create table media_assets (
  id uuid primary key, owner_id text not null references profiles(user_id) on delete
cascade,
  purpose text not null check (purpose in
('avatar','cover','post','highlight','content')),
  status text not null default 'pending' check (status in
('pending','processing','ready','rejected','deleted')),
  storage_key text not null, mime text not null, bytes bigint, width int, height
int,
  blurhash text, thumbhash text, variants jsonb not null default '{}',  --
{"sm":"key","md":"key","lg":"key"}
  created_at timestamptz default now(), ready_at timestamptz
);

-- Notifications
create table notifications (
  id uuid primary key, recipient_id text not null references profiles(user_id) on
delete cascade,
  type text not null,                      --
follow|like|reply|mention|repost|quote|ministry_announcement|system
  group_key text,                          -- e.g. "like:post:<id>" for aggregation
  actor_ids text[] not null default '{}',  -- latest N actors (cap at ~5)
```

```
  actor_count int not null default 1,
  entity_type text, entity_id text, data jsonb not null default '{}',
  read_at timestamptz, created_at timestamptz default now(), updated_at timestamptz
default now()
);
create unique index notifications_group_unique on notifications (recipient_id,
group_key) where group_key is not null and read_at is null;
create index notifications_inbox on notifications (recipient_id, updated_at desc, id
desc);
create index notifications_unread on notifications (recipient_id) where read_at is
null;
create table push_devices (id uuid primary key, user_id text references
profiles(user_id) on delete cascade,
  expo_push_token text not null unique, platform text, app_version text, locale
text, last_seen_at timestamptz, disabled_at timestamptz);
create table notification_preferences (user_id text references profiles(user_id) on
delete cascade, type text,
  push boolean not null default true, in_app boolean not null default true, primary
key (user_id, type));

-- Streaks (server-authoritative, timezone-aware)
create table streak_days (user_id text references profiles(user_id) on delete
cascade, local_date date, created_at timestamptz default now(), primary key
(user_id, local_date));
create table user_streaks (user_id text primary key references profiles(user_id) on
delete cascade, current int not null default 0, longest int not null default 0,
last_local_date date);

-- Fellowship domain
create table ministries (id uuid primary key, slug citext unique not null, name text
not null, description text, cover_asset_id uuid, created_at timestamptz default
now());
create table ministry_members (ministry_id uuid references ministries(id) on delete
cascade, user_id text references profiles(user_id) on delete cascade,
  role text not null default 'member', status text not null default 'active',
joined_at timestamptz default now(), primary key (ministry_id, user_id));
create table fellowship_events (id uuid primary key, ministry_id uuid references
ministries(id), title text not null, description text, location text,
  starts_at timestamptz not null, ends_at timestamptz, recurrence_rule text,  -- RFC
5545 RRULE for weekly schedules
  created_by text references profiles(user_id), created_at timestamptz default
now());
create table highlights (id uuid primary key, owner_id text references
profiles(user_id) on delete cascade, title text not null check (char_length(title)
<= 20),
  emoji text, cover_asset_id uuid, position smallint not null default 0, created_at
timestamptz default now());
create table highlight_items (highlight_id uuid references highlights(id) on delete
```

```
cascade, position smallint, post_id uuid references posts(id) on delete set null,
asset_id uuid, primary key (highlight_id, position));
create table content_items (id uuid primary key, kind text not null default 'video',
title text not null, description text,
  duration_seconds int, thumbnail_asset_id uuid, playback_url text, provider text,
provider_ref text, published_at timestamptz not null, status text default
'published');

-- Safety & plumbing
create table reports (id uuid primary key, reporter_id text references
profiles(user_id) on delete set null, target_type text not null check (target_type
in ('post','profile','media')),
  target_id text not null, reason text not null, details text check
(char_length(details) <= 1000), status text not null default 'open' check (status in
('open','reviewing','actioned','dismissed')),
  handled_by text, handled_at timestamptz, created_at timestamptz default now(),
unique (reporter_id, target_type, target_id));
create table audit_log (id bigserial primary key, actor_id text, action text not
null, target_type text, target_id text, meta jsonb, ip inet, created_at timestamptz
default now());
create table outbox_events (id uuid primary key, type text not null, aggregate_id
text, payload jsonb not null, created_at timestamptz default now(), processed_at
timestamptz, attempts int default 0, last_error text);
create index outbox_unprocessed on outbox_events (created_at) where processed_at is
null;
create table idempotency_keys (user_id text, key text, request_hash text not null,
response_status int, response_body jsonb, created_at timestamptz default now(),
primary key (user_id, key));
```

Modeling rules

- **Soft delete** posts (`deleted_at`) so threads/notifications don't dangle; hard-delete after a retention window via a job. Soft-deleted content must be excluded in *every* list query and serialized as a tombstone only where a thread needs it.
- **Counters** (`followers_count`, `like_count`, …) change **only inside the same transaction** as the edge-row insert/delete, using `UPDATE … SET n = n + 1` guarded by "row actually inserted" (`INSERT … ON CONFLICT DO NOTHING RETURNING`). A reconcile job (§11) repairs drift. Never compute counts with `COUNT(*)` on request for hot paths.
- **Edit window:** posts may be edited for a limited time (e.g. 30 min), keep `edited_at`; optional `post_revisions` table if you need history.
- **Denormalize carefully:** profile card fields embedded in list items are *read* from `profiles` + `media_assets` at hydration time, not copied into posts.
- Add `CHECK` constraints and FKs liberally. The database is your last line of defense.
- **Seed users must exist in Better Auth first** (profiles reference `"user"(id)`).

---

7. API design (REST, `/api/v1`)

7.1 Conventions

- **JSON**, camelCase. **Auth:** session cookie via Better Auth (`requireAuth` unless marked *public*). **Errors:** RFC 9457 problem details.
- **Lists:** `GET …?cursor=<opaque>&limit=20` → `{ items: [...], nextCursor: string | null, hasMore: boolean }`. Max `limit` 50.
- **State toggles are idempotent resources:** `PUT /posts/:id/like`, `DELETE /posts/:id/like` (not `POST /like/toggle`). Same for follow, bookmark, repost, mute, block. Always return the **updated counts/state** so the client can reconcile after an optimistic update.
- **Create endpoints** accept `Idempotency-Key` and return `201` with `Location`.
- **Partial updates:** `PATCH` with an explicit allow-list (no mass assignment). Reject unknown fields.
- **Timestamps:** ISO-8601 UTC strings. The client renders relative times ("3h ago") locally.
- **ETag / `If-None-Match`** on cacheable GETs (profile, ministries, content list) → `304` to save bandwidth.
- **Never** return another user's email, session info, push tokens, IP, or internal flags.

7.2 Canonical response shapes (put in `packages/contracts`)

```
type MediaRef = { id: string; kind: 'image'|'video'; width: number; height: number;
blurhash?: string; thumbhash?: string;
                  urls: { sm: string; md: string; lg: string; original?: string };
alt?: string };

type UserSummary = {               // embedded in EVERY list item so the profile can
open instantly from a seed
  id: string; username: string; displayName: string;
  avatar: MediaRef | null; isVerified: boolean;
  fellowshipRole?: string | null; ministryBadge?: string | null;
};

type Post = {
  id: string; kind: 'post'|'reply'|'repost'|'quote'; author: UserSummary;
  body: string | null; createdAt: string; editedAt?: string | null;
  media: MediaRef[]; replyTo?: { id: string; authorUsername: string } | null;
  repostOf?: Post | null; quoted?: Post | null;          // hydrated one level deep
  counts: { replies: number; reposts: number; likes: number };   // bookmarks stay
private
  viewer: { liked: boolean; reposted: boolean; bookmarked: boolean };
  tombstone?: 'deleted' | 'unavailable';                 // only inside
threads/quotes
};
```

```
type ProfileDetail = UserSummary & {
  cover: MediaRef | null; bio: string | null; location: string | null; joinedAt:
string;
  links: { kind: string; handle?: string; url: string }[];
  academic: { department?: string; studyYear?: number; label?: string };      //
"Software Engineering · 4th Year"
  badges: { kind: 'ministry'|'academic'|'streak'|'role'; label: string }[];   //
drives the chips row
  streak: { current: number } | null;
  counts: { following: number; connects: number; reflections: number };      //
connects = followers (see §1.5)
  presence: { online: boolean; lastSeenAt?: string | null } | null;           //
null when hidden
  viewer: { isSelf: boolean; following: boolean; followedBy: boolean;
pendingRequest: boolean; blocked: boolean; blockedBy: boolean; muted: boolean };
  followedByPreview: { total: number; users: UserSummary[] };                 // up
to 3 mutuals + total
  highlights: { id: string; title: string; emoji?: string; cover: MediaRef | null }
[];
};
```

7.3 Endpoint catalog (build all; mark MVP in phases, §16)

Meta & account

Method & path	Purpose
`GET /meta` *(public)*	`{ minAppVersion, serverTime, features }`
`GET /me`	Viewer profile + unread notification count + streak + settings (powers the home top bar: avatar, flame, bell badge in **one** call)
`PATCH /me/profile`	Edit display name, bio, location, department, year, links, `showPresence`, `visibility`
`PUT /me/username`	Change username (rules + cooldown + reserved words + uniqueness, `409` on conflict)
`POST /me/avatar` · `POST /me/cover`	Attach an uploaded asset (see §8)
`DELETE /me`	Account deletion (§13)
`POST /me/presence`	Heartbeat (rate-limited; updates `last_seen_at`)

Profiles & graph

Method & path	Purpose
`GET /profiles/:username`	`ProfileDetail` (viewer-aware; `404` if blocked-by, per privacy policy)
`GET /profiles/:username/posts?kind=reflections|replies`	Reflections tab
`GET /profiles/:username/media`	Media tab (grid of posts' media)
`GET /profiles/:username/ministries`	Ministry tab: memberships + upcoming events
`GET /profiles/:username/likes`	Likes tab (**privacy rule: self-only by default**; make it a setting)
`GET /profiles/:username/followers` · `/following` · `/mutuals`	Graph lists (paginated, with viewer state)
`PUT /users/:id/follow` · `DELETE /users/:id/follow`	Follow/unfollow (returns new counts + relationship)
`PUT/DELETE /users/:id/block` · `/mute`	Safety
`GET /suggestions/users`	"Who to Follow" (§9)
`DELETE /suggestions/users/:id`	The ✕ dismiss (inserts `follow_dismissals`)

Posts, feed, interactions

Method & path	Purpose
`POST /posts`	Create post/reply/quote (`body`, `mediaAssetIds[]`, `replyToId?`, `quotedId?`)
`GET /posts/:id` · `GET /posts/:id/thread` · `GET /posts/:id/replies`	Detail, ancestors+focused+replies, replies page
`PATCH /posts/:id` · `DELETE /posts/:id`	Edit (window) / soft delete (author or moderator)
`PUT/DELETE /posts/:id/like`	Like/unlike
`PUT/DELETE /posts/:id/bookmark` · `GET /me/bookmarks`	Bookmarks
`PUT/DELETE /posts/:id/repost`	Repost/undo (quote uses `POST /posts`)

Method & path	Purpose
`GET /feed/following` · `GET /feed/for-you`	Feeds (§10)
`GET /feed/head?since=<cursor>`	Cheap "N new posts" check

Media, notifications, search, extras

Method & path	Purpose
`POST /media/uploads`	Create an upload intent → presigned URL + `assetId` (§8)
`POST /media/uploads/:id/complete`	Client signals completion; server verifies & enqueues processing
`GET /media/:id`	Asset status (`pending/processing/ready`) for the composer
`GET /notifications` · `GET /notifications/unread-count`	Inbox (grouped) + badge
`POST /notifications/read`	`{ ids?: string[], all?: true }`
`PUT /me/push-devices` · `DELETE /me/push-devices/:token`	Register/unregister Expo push token
`GET/PATCH /me/notification-preferences`	Per-type push/in-app toggles
`GET /search?q=&type=users|posts|tags` · `GET /search/suggest?q=`	Search
`GET /trending/tags`	Trending hashtags (rolling window)
`POST /streak/check-in` · `GET /streak`	Streak (idempotent per local day)
`GET /content/videos` · `GET /content/videos/:id`	Home video list
`GET /ministries` · `GET /ministries/:slug` · `PUT/DELETE /ministries/:slug/membership`	Ministries
`GET /events?from=&to=`	Schedule
`GET/POST/PATCH/DELETE /me/highlights…`	Highlights CRUD
`POST /reports`	Report post/profile/media

Method & path	Purpose
`GET /admin/reports` · `PATCH /admin/reports/:id` · `POST /admin/users/:id/verify` · `…/suspend`	Admin/moderator (role-checked, audited)

7.4 Authorization policy layer (single source of truth)

Implement `canViewProfile(viewer, target)`, `canViewPost(viewer, post)`, `canInteract(viewer, post)`, `canMessage…` in `policy.ts` files and **mirror them as SQL predicates** for list queries (a `visiblePostsPredicate(viewerId)` helper). Rules:

- Soft-deleted/suspended content: invisible (except to moderators).
- **Blocks are symmetric in visibility:** if A blocks B, neither can see the other's posts/profile in lists, and B cannot follow/interact with A. Return `404`, not `403`, to avoid confirming existence.
- `visibility = 'followers'` content is visible only to accepted followers and the author.
- **IDOR defense:** never accept `userId` for "me" actions from the body. Derive from the session. For resources by id (`/posts/:id`), load, then check policy, then act.
- Authorization tests are mandatory (§17): a matrix of viewer × resource × action.

---

8. Media pipeline (avatars, covers, post images, highlights)

Goal: fast, safe uploads that never block the API, and images that render with **zero layout shift** and instant placeholders (this is what makes the profile zoom transition and long-press preview look native).

8.1 Upload flow (presigned, direct-to-storage)

1. App → `POST /media/uploads` `{ purpose, mime, bytes, width?, height? }` (with `Idempotency-Key`).
   Server validates: purpose allow-list, MIME allow-list (`image/jpeg|png|webp|heic`), **size caps** (avatar 5 MB, post image 10 MB), per-user daily quota, then creates `media_assets(status='pending')` and returns `{ assetId, uploadUrl, headers, expiresAt }` . Presigned URL: short TTL (5 to 10 min), **content-type and content-length conditions** pinned, key = `u/{userId}/{assetId}/original` (never user-supplied filenames).
2. App uploads the bytes straight to the bucket (`PUT`) using `expo-file-system` 's upload API (supports progress and background) or `fetch` . Show a progress bar in the composer.
3. App → `POST /media/uploads/:assetId/complete` . Server `HEAD` s the object, checks size/type, sets `status='processing'` , enqueues a **media job**.
4. **Worker (sharp):** sniff the real file type (don't trust the client's MIME), reject on mismatch/oversize/decompression-bomb limits (`sharp` `limitInputPixels`), **auto-rotate, strip EXIF (especially GPS)**, generate variants (e.g. `sm` 96 px, `md` 480 px, `lg` 1080 px, WebP/AVIF +

JPEG fallback), compute `blurhash` / `thumbhash`, store width/height, set `status='ready'`. Failures → `rejected` with a reason.

5. App polls `GET /media/:id` (or receives the status in the notification/WS) until `ready`, then submits the post with `mediaAssetIds`. **A post cannot reference a non-ready asset owned by someone else** (check owner + status in the create transaction).
6. **Orphan cleanup job:** assets `pending` > 24 h or not attached to any post/profile/highlight for > 48 h → delete object + row.

8.2 Serving

- Public CDN domain in front of the bucket, `Cache-Control: public, max-age=31536000, immutable` on variant keys (they're content-addressed by asset id + variant).
- API returns **absolute URLs** per variant plus `width`, `height`, `blurhash/thumbhash`. The app passes the placeholder to `expo-image` and reserves the exact aspect ratio in the layout.
- **Avatar variants are mandatory:** `sm` (≥ 96 px, for lists), `md` (≥ 300 px, profile header), `lg` **(≥ 1000 px, long-press preview)**.
- Private/ `followers` content media: use **signed CDN URLs** with expiry, or keep the bucket private and proxy via short-lived signed URLs.

8.3 Video

- The home "videos" list is **curated content** (`content_items`). Use a managed provider (Mux/Cloudflare Stream) or store external playback URLs. Store provider id, duration, and a thumbnail asset. Don't build transcoding. User-generated video can wait; if added later, use the provider's direct-upload flow and webhooks.

---

9. The social graph, suggestions, presence, search

9.1 Follow semantics

- `PUT /users/:id/follow` runs in **one transaction**: reject self-follow and blocked pairs → `INSERT … ON CONFLICT DO NOTHING RETURNING` → **only if a row was inserted** increment `following_count` (follower) and `followers_count` (followee) → write outbox `follow.created` → return the new relationship + counts. `DELETE` mirrors it. Private accounts (`visibility='followers'`, if enabled) use `status='pending'` and an approval flow; counts increment on accept.
- **Blocking** removes follows in both directions (and fixes counters) in the same transaction.
- Anti-abuse: cap follows/hour and total following per day; follow-churn detection (follow/unfollow loops don't spam notifications: de-duplicate `follow` notifications per actor per 24 h).

9.2 "Followed by X, Y and N others" (profile) and "N mutual peers" (suggestion cards)

```
-- Mutuals for a profile P as seen by viewer V: people V follows who follow P
select f2.follower_id
from follows f1                                -- V follows U
join follows f2 on f2.follower_id = f1.followee_id and f2.followee_id = $P   -- U
follows P
where f1.follower_id = $V and f1.status='accepted' and f2.status='accepted'
order by f2.created_at desc limit 3;          -- preview users
-- total = count(*) of the same join (cap the count at e.g. 99 for speed or maintain
a cache)
```

Return `followedByPreview: { total, users[0..3] }`. For the suggestion carousel compute `mutualCount` in **one set-based query for all candidates** (join on `followee_id = ANY($candidateIds)`), never per candidate. Cache results in Redis for a few minutes keyed by `(viewerId, candidateId)` or per viewer page.

9.3 "Who to Follow" (candidate generation → filter → rank)

Candidates (union, each with a source tag and a score contribution):

1. **Friends-of-friends:** followees of the people I follow, ranked by number of mutuals (strong signal).
2. **Same ministry:** members of ministries I'm in.
3. **Same department and year** (campus affinity).
4. **Popular in the fellowship:** high recent follower growth / engagement (cold-start fallback for brand-new users).
5. **People who follow me but I don't follow back.**

Filters: exclude self, already-followed, blocked/blocking, muted, **dismissed (`follow_dismissals`)**, suspended, unverified-email accounts if you require verification, and users with `visibility`/privacy settings that opt out of suggestions.

Rank: `score = 3·mutuals + 2·sameMinistry + 1·sameDept + 1·sameYear + 0.5·followsMe + popularityNorm + recencyOfActivity`, then **diversify** (don't return 10 people from the same ministry) and add light randomization (seeded per viewer per day) so the carousel feels fresh but is stable within a session. Return: `UserSummary` + `fellowshipRole`, `academicLabel` ("Software Eng. • 4th Yr"), `mutualCount`, `reason` (`"mutual"|"ministry"|"department"|…`) and `presence.online`.

Endpoints: carousel = `GET /suggestions/users?limit=10`; "View all" = same endpoint with cursor pagination; ✕ = `DELETE /suggestions/users/:id`; after Follow, the card is removed client-side and the list is refetched lazily.

9.4 Presence (the green dot)

- App sends `POST /me/presence` every ~60 s **while foregrounded** (and on app start/stop). Server sets `profiles.last_seen_at = now()` (rate-limit to 1 write / 30 s per user; use Redis `SET`

`presence:{id} EX 120` as the hot store and flush to Postgres periodically to avoid write amplification).

- `online = showPresence && now - lastSeen < 2 min`. Respect `show_presence=false` (return `null`). Don't leak presence of blocked users.
- Lists include `presence` only where the UI shows it (suggestions, followers list).

9.5 Search

- **People:** `ILIKE` /trigram on `username` and `unaccent(lower(display_name))` using the GIN trigram indexes; rank exact/prefix first, then similarity, then follow-graph proximity (people I follow rank higher).
- **Posts:** Postgres full-text search (`to_tsvector('simple', body)` + `websearch_to_tsquery`) with visibility/block filters, recency-weighted. (Use the `simple` config unless you add language-specific handling. Amharic isn't covered by built-in stemmers.)
- **Hashtags:** parse `#tags` on create into `hashtags/post_hashtags`; `GET /trending/tags` = tag counts in the last 24 to 48 h (cached 5 min).
- **Autosuggest** `GET /search/suggest?q=` returns top users + tags, **debounced** on the client, with a minimum 2-char query and tight rate limit.
- Add Meilisearch/Typesense only if Postgres search measurably can't keep up.

---

10. Posts, threads, feeds, and the rest of the content domain

10.1 Creating posts (the write path)

`POST /posts` (Idempotency-Key) in one transaction: validate (length ≤ 500 grapheme-aware, body-or-media required, max 4 media, `replyToId` / `quotedId` visible to the viewer and not blocked) → insert post (set `root_id` for replies) → link media (verify ownership/ `ready`) → parse mentions & hashtags (resolve usernames → `post_mentions`; **mentions of users who blocked the author are ignored**) → increment `posts_count` (for non-replies) and the parent's `reply_count` / quoted post's `quote_count` / reposted post's `repost_count` → outbox `post.created` (feeds fan-out, notifications for mentions/replies/quotes, search). Return the hydrated `Post`.

- **Edit:** within the edit window only, re-parse mentions/hashtags, set `edited_at`. **Delete:** soft delete + decrement counters + enqueue cleanup (remove from timelines/notifications/search).
- **Reposts:** `PUT /posts/:id/repost` creates a `kind='repost'` row (unique per user/original). Reposting a repost resolves to the original. A repost shows in followers' feeds as "Abenezer reposted" with the original post embedded.
- **Content rules:** profanity/spam heuristics (rate + duplicate-content + link-count checks), per-user post limit (e.g. 50/day), shadow-limit on new accounts. Hook for a moderation provider later.

10.2 Thread view

`GET /posts/:id/thread` → `{ ancestors: Post[] (root → parent), focus: Post, replies: { items, nextCursor } }`. Replies ordered by `created_at asc` (or "top" by like_count later). Use `root_id` to fetch an entire conversation efficiently. Deleted ancestors/replies appear as **tombstones** so threads don't break.

10.3 Feeds (the read path)

Following feed (`/feed/following`): posts by people I follow + my own, excluding replies, blocked/muted authors, deleted, and visibility-restricted content; **reposts** surface the original (dedupe: if both original and repost appear, show once, keeping the earliest relevant time).

```
select p.id, p.created_at
from posts p
where p.deleted_at is null
  and p.kind in ('post','repost','quote')
  and (p.author_id = $viewer or p.author_id in (select followee_id from follows
where follower_id = $viewer and status='accepted'))
  and not exists (select 1 from blocks b where (b.blocker_id=$viewer and
b.blocked_id=p.author_id) or (b.blocker_id=p.author_id and b.blocked_id=$viewer))
  and not exists (select 1 from mutes  m where m.muter_id=$viewer and
m.muted_id=p.author_id)
  and (p.created_at, p.id) < ($cursorTs, $cursorId)
order by p.created_at desc, p.id desc
limit $limit + 1;
```

Two-step: **(1) fetch ordered ids**, **(2) hydrate** by `id = ANY($ids)` (authors, media, counts, repost/quote embeds, viewer state with one query each for likes/bookmarks/reposts) and restore order. `limit+1` tells you `hasMore`.

Scaling model (explain it in an ADR): at campus scale, **fan-out-on-read** (the query above) with the `posts_author_time` index is fast and simple. Put the feed behind a `FeedProvider` interface so you can later switch to **fan-out-on-write** (`timeline_entries(user_id, post_id, created_at)` filled by the outbox worker, with a hybrid "pull for high-follower accounts" rule) *only if* measurements show p95 > target. Add the measurement (k6 script + `EXPLAIN` output) to the ADR.

For You feed (`/feed/for-you`): candidate generation (recent posts from followed users + posts from friends-of-friends + high-engagement recent posts from the fellowship) → filter (same predicates as above + dismissed/ `not interested`) → **rank**: `score = log1p(likes·1 + replies·2 + reposts·3) · recencyDecay(age, halfLife≈12h) · affinity(author)` + a small exploration term → dedupe/diversify (max 2 consecutive posts per author) → hydrate. Keep ranking in one pure, unit-tested function so it's explainable (`debug=1` for admins returns score components). Use a **stable per-session seed + cursor** that encodes the rank snapshot so pagination doesn't reorder.

New-post indicator: `GET /feed/head?since=<newest cursor>` returns `{ newCount }` (cheap `count` capped at 50) for the "N new posts" pill; the client refetches page 1 on tap or pull-to-refresh.

10.4 Counters, likes, bookmarks (hot path)

- `PUT /posts/:id/like`: `INSERT … ON CONFLICT DO NOTHING RETURNING` → if inserted: `like_count+1`, outbox `like.created` (notification for the author, **de-duped/aggregated**: "Hiwot and 3 others liked your reflection"). `DELETE` mirrors without notification. Response: `{ liked: true, likeCount }`.
- Bookmarks are **private** (never visible to others, count not exposed). Bookmark state is part of `viewer`.
- Likes tab privacy: default **only the owner sees their likes** (configurable).

10.5 Fellowship domain: ministries, events, highlights, content library

- **Ministries/events:** `GET /profiles/:username/ministries` returns memberships (name, role, "Active" status) and the user's upcoming events (expand `recurrence_rule` server-side for the next N occurrences in the user's timezone; store RRULE, don't store every occurrence). Admin/leaders manage ministries/events (role checks). Event reminders connect to the app's local-reminders feature via `GET /events` data; optionally push "starting soon" via a scheduled job.
- **Highlights:** CRUD for the owner (max ~10 highlights, ≤ 20 items each). Items reference posts or assets; a cover asset is required. Deleting a post nulls the item (`ON DELETE SET NULL`) and a cleanup job removes empty highlights.
- **Content library (home videos):** `GET /content/videos?cursor=` ordered by `published_at desc`; admin-curated; list item = `{ id, title, description, durationSeconds, thumbnail: MediaRef, playbackUrl? }`. Cache with ETag + short TTL. Watch progress (optional later).

10.6 Streaks (the flame chip and "32d Streak" badge)

- **Definition (put it in an ADR):** a *qualifying action* (daily check-in; optionally opening the app, reading a devotional, or posting) counts for the **user's local calendar date** (`profiles.timezone`, default `Africa/Addis_Ababa`).
- `POST /streak/check-in` is **idempotent per local day**: compute `localDate` on the **server** from `now()` + timezone (never trust a client date), `INSERT INTO streak_days … ON CONFLICT DO NOTHING RETURNING`; if inserted, update `user_streaks`: `current = (last_local_date == localDate - 1) ? current + 1 : 1`, `longest = max(longest, current)`, `last_local_date = localDate`. Return `{ current, longest, checkedInToday }`.
- Reading the streak: if `last_local_date < localToday - 1` then the displayed current streak is **0** (broken) even before the nightly rollup. Optional **grace/freeze** token as a later feature.
- DST isn't an issue in Ethiopia, but use a proper tz library (`Intl` / `date-fns-tz` / `luxon`) so other timezones work.

- Nightly job: persist broken streaks, award milestone notifications (7/30/100 days).

---

11. Notifications, push, and background jobs

11.1 In-app notifications (bell badge + inbox)

- Created **only by workers** consuming outbox events (never inline in request handlers). Types: `follow`, `like`, `reply`, `mention`, `repost`, `quote`, `ministry_announcement`, `streak_milestone`, `system`.
- **Aggregation:** like/repost/follow events upsert on `(recipient_id, group_key)` (e.g. `like:post:<id>`), push the actor into `actor_ids` (cap 5), bump `actor_count`, set `updated_at`, and clear `read_at` only if materially new. UI: "Hiwot, Bethelhem and 26 others liked your reflection".
- **Skip** notifications for self-actions, blocked/muted actors, and users who disabled the type.
- `GET /notifications` (cursor on `(updated_at, id)`) hydrates actors/entities in batch. `GET /notifications/unread-count` is O(1)-ish via the partial index (and cached in Redis; bust on write/read). `/me` includes the same count so the bell badge costs no extra request.
- `POST /notifications/read` marks specific/all as read; opening the inbox shouldn't auto-mark everything read unless that's the product choice.

11.2 Push (Expo Push Service)

1. App registers: `getExpoPushTokenAsync` → `PUT /me/push-devices { token, platform, appVersion, locale }`. Upsert by token; **re-assign** the token if it's now used by a different user (shared/test devices); remove on sign-out.
2. Worker builds messages (title/body/`data: { url: '/profile/…' }` deep link/`channelId: 'social'` / `categoryId`), batches ≤ 100 per request via `expo-server-sdk`, respects **preferences + quiet hours (user timezone)**, collapses bursts (**don't push 30 likes: send one aggregated push per post per 10 min**).
3. **Tickets & receipts:** store tickets, poll receipts after ~15+ min; on `DeviceNotRegistered` mark the device `disabled_at` and stop sending. Retry transient errors with exponential backoff.
4. Android channel(s) must exist on the client (the reminders guide already creates one; add a `social` channel with a lower importance than reminders).
5. Foreground handling on the client: show an in-app banner and bump the unread badge; tapping deep-links via `data.url` (reuse the existing notification response handler).
6. Push is **separate from local reminders** (offline-first, already built). Don't mix the channels or the preferences.

11.3 Background jobs (BullMQ)

Queue/job	Trigger	Notes
`outbox-relay`	every 1 to 2 s poll	`SELECT … FOR UPDATE SKIP LOCKED`, at-least-once, idempotent consumers
`notify`	outbox events	create/aggregate notifications, enqueue push
`push-send` / `push-receipts`	notify / delayed	tickets → receipts → prune tokens
`media-process`	upload complete	§8
`media-orphans`	hourly	delete unattached assets
`counters-reconcile`	nightly	recompute `followers/following/posts/like/reply/repost` counts from source tables in batches; log + alert on drift
`streak-rollup`	hourly per timezone	break streaks, milestones
`search-index` (only if external engine)	outbox	
`retention`	nightly	hard-delete soft-deleted content past retention, expired idempotency keys, old receipts/outbox
`account-delete`	user request	§13

All jobs: idempotent, retried with backoff, dead-letter queue, structured logs with job id, and metrics (queue depth, age of oldest job, failure rate).

11.4 Real-time (optional, later)

Start with **pull + push + refetch-on-focus**. If product wants live updates, add **SSE** (simple, one-way) for `notification.created`, `feed.new` and counters on the open screen, backed by Redis pub/sub. Don't add WebSockets until you need bidirectional features (DMs/typing).
